Trap, Forward, Resume: Capability Revocation as the Read Barrier of a Moving Garbage Collector on CHERI

XIAOHUI LUO*, Independent Researcher, China

Concurrent moving garbage collectors pay for relocation with a read barrier: a check on reference loads that redirects stale references to relocated objects. Production collectors implement the barrier in software—ZGC's colored-pointer test, Shenandoah's forwarding pointer—or, in one famous case, in custom silicon. CHERI capability hardware offers a commodity-adjacent alternative that has not been exploited: *capability revocation*, designed for C/C++ temporal memory safety, makes the load–store unit itself fault on any use of a revoked pointer. We repurpose this safety mechanism as a moving collector's read barrier. Our collector, StoplessGC, relocates objects, revokes the capabilities to the old copies, and lets the hardware trap each surviving stale reference; a signal handler forwards the faulting register through a forwarding table and resumes the load. No check is executed on the hot path of reference loads—the barrier is the capability tag check the load–store unit performs on every purecap access anyway.

We implement StoplessGC in the OpenJDK 17 template interpreter on Arm Morello (pure-capability CheriBSD) and measure it under QEMU system emulation. The relocation pause is bounded by root-slot count plus relocated bytes and is independent of heap size: with the root set held constant, the pause varies by 7% while the heap footprint grows 46× (the prototype never reclaims, so this is allocated, not live, memory). The dominant cost is the whole-address-space revocation sweep, which is linear in memory size; because the sweep's cost is approximately independent of how much it revokes, batching relocations across collection cycles amortises the mean cycle pause by an observed 10.3× at batch size 8 in a workload-constrained stop-the-world configuration—though we show the batch window is unsound for general concurrent programs (stale-but-still-tagged capabilities break reference identity), and we verify that the hardware primitive that closes the window, the per-page capability-load generation barrier, works under our stack: opening a revocation epoch costs milliseconds and lazily revokes on first capability load. What we build and measure is the relocation and barrier substrate of a moving collector (liveness, reclamation, and address reuse are future work, and we are explicit about that boundary). We also report the design laws capability hardware forces on such a collector—forwarding metadata must be integers re-derived from revocation-exempt roots, and lookups must chase forwarding chains transitively, because freshly derived capabilities to stale intermediate copies are valid—and a taxonomy of the pure-capability porting hazards we hit in HotSpot.

1 Introduction

A moving garbage collector must ensure that no mutator dereferences a stale reference to an object that has been relocated. Concurrent collectors enforce this with a *read barrier* [2]: extra work on reference loads that detects stale references and redirects them. The engineering history of low-latency Java collectors is largely a history of making this barrier cheap. Azul's Pauseless collector put it in custom hardware [5]; C4 re-implemented it in software on x86 [13]; Shenandoah dereferences a per-object forwarding pointer [4, 8]; ZGC tests metadata bits embedded in the pointer itself [18]. In every case the mutator executes instructions on the hot path of reference loads that exist only for the collector's benefit.

CHERI [16] and its Arm Morello implementation [9] change the substrate. In pure-capability ("purecap") code, every pointer is a 128-bit capability with hardware-checked bounds, permissions, and an out-of-band validity tag; every load and store is checked by the load–store unit at no software cost. On top of this, CheriBSD provides *capability revocation* [6, 17]: mark an address range in a

*The system and this manuscript were produced by Claude (Anthropic)—Opus 4.8 and Fable 5 models, via Claude Code—working autonomously under the direction and review of the human author, who takes full responsibility for the content. Current publisher and arXiv policies do not permit listing an AI system as an author; this note and the statement at the end of the paper are the prominent disclosure those policies require.

Author's Contact Information: Xiaohui Luo, Independent Researcher, China, lmhtq1991@gmail.com.

shadow bitmap, and a kernel sweep atomically invalidates every capability in the address space that points into the marked range. Subsequent use of a revoked capability traps. Revocation was designed to give C and C++ heaps temporal memory safety—to make use-after-free fail-stop—and its costs and mechanisms have been studied in that setting [6, 7, 17].

Our observation is that revocation is, almost verbatim, the read barrier of a moving collector. The difference is what happens at the trap. In the temporal-safety setting a revoked capability is a use-after-free; the program is wrong and stops. In a moving collector a revoked capability is merely *stale*; the object lives on at a new address. If the trap handler can map the old address to the new object, it can repair the faulting register and resume the load. The hardware provides exact, per-capability staleness detection on every dereference; the collector only pays, lazily, for references that are actually used.

We built this collector. StoplessGC[1] relocates Java objects with a capability-preserving copy, records *from*→*to* in a forwarding table, revokes the old copies, and heals each trapped mutator access in a `SIGPROT` handler. To our knowledge it is the first garbage collector to use capability revocation as the read barrier of a moving collector—prior CHERI revocation work invalidates pointers but never forwards them, and prior Java-on-CHERI work ports only non-moving or stop-the-world-compacting collectors [14].

This paper makes the following contributions:

- **Mechanism** (§3): the trap–forward–resume protocol; an identity barrier for reference equality, the one operation that must observe relocation but does not dereference; and batched revocation, which exploits the fact that the sweep's cost is independent of the number of objects revoked.
- **A capability-specific design law** (§3.2): the collector's forwarding metadata must be stored as plain integers, not capabilities. The revocation sweep scans *all* memory—including the collector's own tables—and will destroy forwarding entries that point into ranges revoked by a later cycle. We learned this from a corruption bug whose signature we describe so others can recognise it.
- **Implementation** (§4): StoplessGC in the OpenJDK 17 template interpreter on purecap CheriBSD/Morello, ∼2.6 kLOC of new runtime plus interpreter patches, with a taxonomy of the purecap porting hazards that cost us the most time.
- **Measurement** (§5): with the root set held constant, the relocation pause varies 7% while the live heap grows 46× (3.6 → 163 MiB)—the pause is bound to root slots plus relocated bytes, not heap size. The revocation sweep is linear in memory size and dominates total cost; batching amortises the mean cycle pause by an observed 10.3× at batch size 8 (workload-constrained stop-the-world configuration). Healing a trapped reference costs ∼20 µs of in-handler work under emulation, and the identity barrier's slow path executes only hundreds of times per run.

We are explicit about scope (§6). What is built and measured is the *relocation and barrier substrate* of a moving collector: relocation, revocation, healing, and identity, validated by integrity tests that relocate every directly root-referenced object (1,016 in the largest configuration) and verify structure and identity afterwards. It is not yet a complete collector—there is no liveness analysis (relocation candidates come from the root scan), no reclamation of dead objects, and no address reuse (§3.2)—and the evaluation runs under QEMU system emulation, so absolute times are inflated and only the *shapes*—flat versus linear, the amortisation ratio—are architectural claims. Relocation is stop-the-world in the measured configuration (a concurrent collector thread is implemented and

[1]The name is an homage to, not a claim of lineage from, the Stopless real-time collector of Pizlo et al. [12], which addresses lock-free concurrent compaction on conventional hardware.

stable under light load), and execution is interpreter-only, as JIT compilation on Morello remains future work across the ecosystem. We believe the substrate is the research question—liveness and reclamation compose with it by known techniques—but the reader should weigh the results as mechanism measurements, not end-to-end GC benchmarks.

2 Background

2.1 CHERI, Morello, and purecap execution

A CHERI capability is a 128-bit pointer carrying a 64-bit address, compressed bounds, permissions, and a 1-bit validity tag held out-of-band by the memory subsystem [16]. Capabilities can only be derived from other capabilities by monotonic restriction (narrow the bounds, drop permissions); any attempt to forge or widen one clears the tag, and dereferencing an untagged capability traps. In purecap code every pointer—every Java object reference included—is a capability, and ordinary loads and stores are capability-checked by hardware. Morello [9] is Arm's experimental implementation; CheriBSD [15] is the FreeBSD-derived OS that runs purecap userspace.

2.2 Capability revocation

Revocation gives CHERI heaps temporal safety. CHERIvoke [17] established the sweeping-revocation design: quarantine freed memory, mark it in a shadow bitmap, then sweep memory and clear the tag of (or strip the permissions of, depending on kernel build) every capability pointing into marked ranges. Cornucopia [6] implemented this in CheriBSD with kernel support, and Cornucopia Reloaded [7] nearly eliminated its stop-the-world pauses using Morello's per-page capability-load generation barrier. CheriBSD exposes the mechanism to userspace as a shadow bitmap plus a `cheri_revoke` system call. Throughout this line of work the post-revocation trap is a *verdict*—the access is a temporal-safety violation. No forwarding is needed because freed objects have no successor.

2.3 Read barriers in moving collectors

Baker's incremental collector [2] introduced the read barrier; Brooks [4] made it an unconditional indirection through a forwarding pointer. Appel, Ellis, and Li [1] moved the check into hardware a generation early, using virtual-memory page protection: protect from-space pages and let the fault handler evacuate. Its granularity problem—one page-sized fault stalls every object on the page, and protection changes require TLB shootdowns—foreshadows our per-capability contrast. Azul's Pauseless collector [5] added a dedicated read-barrier instruction to the Vega processor; C4 [13] brought the same loaded-value-barrier design to x86 in software. ZGC encodes GC state in unused high bits of the 64-bit pointer ("colored pointers") and tests them on load, using multiple virtual mappings of the heap to make the colored address dereferenceable [18]. Shenandoah [8] stores a forwarding pointer in the object header.

2.4 Java on CHERI, and why ZGC's encoding does not transfer

The MOJO project ported OpenJDK 17 to purecap CheriBSD/Morello, including the Epsilon, Serial, and G1 collectors [14]; the concurrent moving collectors (ZGC, Shenandoah) were not ported. This is not an accident of effort: ZGC's two structural optimisations are specifically hostile to capabilities. Colored pointers require unused, software-controlled address bits, but a capability's address must stay within its bounds and cannot carry out-of-band metadata—and the pointer is 128 bits with a hardware-owned tag, so "just use the high bits" forges an invalid capability. Multi-mapping requires the same physical object to be reachable at several virtual addresses, but a capability is bound to the address range it was derived for; dereferencing the same object through a differently-colored

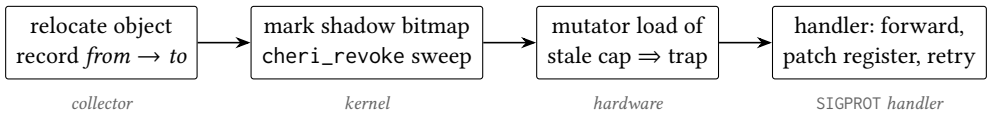

Fig. 1. The trap–forward–resume protocol. The mutator's fast path contains no barrier code: staleness detection is the capability tag check the load–store unit performs on every purecap access anyway.

alias requires re-deriving capabilities at every color flip. A capability-native moving collector needs a different barrier mechanism, which is what this paper supplies.

3 Design

3.1 Overview

StoplessGC manages a single contiguous arena. Allocation derives a tight-bounds capability for each object from the arena's root capability (`csetbounds` to the object's exact size, software permissions stripped). A collection cycle (Figure 1):

(1) **Relocate.** At a safepoint, scan the strong roots (thread stacks, OopStorage, class-loader data). Each root's referent is copied to a fresh arena slot with a capability-preserving copy (the copy must use capability loads/stores, or every reference field in the object loses its tag), the pair (*from*, *to*) is recorded in the forwarding table, and the root slot is updated.
(2) **Revoke.** The old object ranges are marked in the CheriBSD revocation shadow bitmap and a `cheri_revoke` sweep is issued. After the sweep, every surviving capability to an old copy—heap fields, VM-internal slots, registers—is invalid.
(3) **Heal lazily.** When a mutator dereferences a stale reference, the access traps (`SIGPROT`). The handler extracts the faulting capability from the saved register file, looks up its *base address* in the forwarding table, rebuilds a valid capability to the new copy, writes it back to the faulting register, and returns; the instruction retries and succeeds. References that are never used are never healed and cost nothing.

Unlike ZGC's self-healing barrier, the handler cannot repair the memory slot the stale reference was loaded from: the trap fires at *use* time, after the reference was loaded, and the source slot's address is no longer identifiable. Only the faulting register is patched. A heap field holding a stale reference therefore re-traps on every reload until the program itself overwrites the field (or a later cycle's fixup pass reaches it); the cost is bounded by the per-heal cost (§5.3) per reload, not amortised away. This is a structural asymmetry of use-time barriers versus load-time barriers, and the honest price of getting the barrier for free.

3.2 The forwarding table must not contain capabilities

The single most consequential design point is what the forwarding table stores. The obvious representation—map `from_addr` to a capability for the new object—is wrong, and fails in a way specific to capability hardware.

The revocation sweep scans *all* of memory, including the collector's own metadata. Suppose object A is moved to A' in cycle i (table entry $A \to \mathrm{cap}(A')$), and A' is itself moved in a later cycle j and its range revoked. The sweep finds the capability $\mathrm{cap}(A')$ *inside the forwarding table* and clears its tag. A mutator that still holds a stale reference to A then traps, the handler looks up A—and hands back an untagged capability. The mutator faults again, this time unforwardably. The failure is non-deterministic (it needs an object to move twice with a live stale reference across both moves)

and presents as inexplicable corruption at the start of unrelated cycles; disabling revocation makes it vanish, which falsely implicates the sweep itself.

The fix follows from the diagnosis: store plain integers—destination address and length—which the sweep cannot touch, and rebuild the capability at lookup time from the arena's root capability: `csetbounds(csetaddr(arena_root, addr), len)`, with software permissions stripped. Two details are load-bearing. First, the arena root carries CheriBSD's `SW_VMEM` software permission, which exempts it from revocation, so the rebuild source survives every sweep. Second, the rebuilt capability must reproduce the allocator's *exact tight bounds*: both the interior-pointer heal and the identity barrier (below) key their lookups on `cheri_base`(c), the object's base address. An early version that rebuilt arena-wide capabilities broke reference identity in `java.lang.invoke`'s identity-keyed caches, far from the collector. The general law: *on revocation-based systems, GC metadata that must survive revocation must be integers, with capabilities re-derived on demand from a revocation-exempt root.*

Forwarding entries persist across cycles, and lookups must chase them *transitively*: when $A \to A'$ and later $A' \to A''$, a handler that resolves a stale reference to A by one table hop would derive a *fresh, valid* capability to A'—revocation invalidates the capabilities that exist at sweep time, it does not poison the address range—and the mutator would silently read the stale intermediate copy, with no second fault to save it. The lookup therefore follows the integer chain to the final address before deriving a capability (the chain is acyclic because addresses are never reused; a depth guard exists for table-corruption paranoia and *fail-stops* rather than returning a partial hop, since a partial hop is a silent stale read). We state this explicitly because we initially got it wrong: the intuitive "one fault per generation" model only holds if the first heal races between the two sweeps. It is the second instance of the same design law—*a freshly derived capability is always live; correctness must come from the integer metadata, never from expecting revocation to re-trap.*

Address reuse. The prototype's arena is a bump allocator that never reuses addresses, so a forwarding key denotes one object generation forever. Reuse breaks that: if address X held A (moved, entry $X \to \cdot$) and is later reallocated to B which is also moved, the table would need two entries keyed X, and a trapped capability with base X would be ambiguous between generations. Reuse therefore requires an invariant the prototype does not yet implement: a region may be recycled only once no unhealed reference into its previous generation can remain (an epoch scheme, a forced heal pass over surviving stale capability stores, or relocation-set draining as in ZGC's per-cycle remap). We flag this as the main gap between the relocation substrate measured here and a complete collector (§6).

3.3 The heal handler

The handler receives the kernel's capability-fault signal and must decide: stale (forward it) or genuinely bad (a real bug; re-raise). The forwarding table itself is the oracle—only addresses the collector relocated are in it. Three mechanics matter:

- **Register identification.** CheriBSD delivers the faulting PC, not the faulting capability. The handler scans the saved capability register file for registers that are untagged (or permission-stripped—CheriBSD kernels represent revocation either way, and the handler must accept both) and whose address forwards.
- **Interior pointers.** A faulting capability may address a field or array element. Because allocation bounds are tight, the object's base is `cheri_base` of the faulting capability; the handler looks up the base and re-applies the offset to the rebuilt capability.

- **Coexistence with expected faults.** HotSpot deliberately takes faults (the `SafeFetch` probes); on purecap these arrive as capability faults too, and every handler in the chain—including crash dumpers—must replay that redirect before treating a fault as fatal, and must itself attempt the heal path.

3.4 The identity barrier

Reference equality (`if_acmpeq/ne`, `==`) observes addresses without dereferencing, so the hardware barrier never fires: comparing a stale capability to a healed one yields *false* for the same object. ZGC solves this in the load barrier (references are healed before comparison); we must solve it at the comparison. The interpreter's compare becomes three-tier: (1) equal addresses ⇒ equal (no relocation can make two objects share an address); (2) both capabilities tagged ⇒ plain comparison is correct—*on kernels built with* `CHERI_CAPREVOKE_CLEARTAGS` (ours is), where revocation clears tags; CheriBSD's alternative representation strips permissions and leaves the tag set, and would require the fast path to also test permissions; (3) otherwise, call a runtime that normalises both operands through the forwarding table and compares the results. The fast paths add two `gctag` instructions and a branch to one bytecode; the slow path is rare (§5.5).

The same hazard recurs wherever the runtime observes reference *values* without dereferencing, and the bytecode comparison is only the most frequent case. Reference CAS (`Unsafe/VarHandle/AtomicRefere` compares capability bit patterns and can fail spuriously against a healed value; JNI `IsSameObject` and VM-internal address-keyed tables are analogous. (Identity hash codes are safe: HotSpot stores them in the header on first use, and relocation copies the header.) Our implementation covers the interpreter comparison bytecodes and the VM paths our workloads exercise; we make no completeness claim, and a production system would need an audit of every value-mode use of references.

3.5 Batched revocation

The revocation sweep's dominant cost—scanning all mapped memory for capabilities—is approximately independent of how many objects are marked. The shadow bitmap accumulates marks, and forwarding entries already persist and chain across cycles. So the collector can relocate every cycle but sweep every Nth cycle, amortising the dominant cost by up to N. The trade-off is a window in which old copies remain live, and it is unsound for general programs in two distinct ways. First, *mutation*: a not-yet-healed reference can read (or write) the old copy while healed references use the new one, and the copies diverge. Second, and more insidiously, *identity*: until the sweep runs, stale capabilities are still *tagged*, so the identity barrier's both-tagged fast path (§3.4) trusts them and judges the old and new copies of the same object unequal. Identity-sensitive code running inside the window—HotSpot's `java.lang.invoke` bootstrap is dense with identity-keyed caches—fails nondeterministically depending on where collection cycles land, and we observe exactly that in practice (§5.4). Batching as implemented is therefore a measurement of the amortisation mechanism, not a deployable configuration; a sound design must keep the window closed to mutators, for which Morello's per-page capability-load generation barrier—used by Cornucopia Reloaded [7] for exactly this window problem in free–reuse—is the natural instrument.

4 Implementation

StoplessGC is implemented against OpenJDK 17 (jdk17u) on purecap CheriBSD 15 (`aarch64c`). The collector proper is a CHERI-side runtime library (~1.6 kLOC of C: forwarding table, revocation glue, signal handler) plus a HotSpot `CollectedHeap` implementation (~1 kLOC of C++: arena, root-scan closures, the collection `VM_Operation`, an optional concurrent collector thread). Around 170 further patches against HotSpot make the template interpreter and runtime capability-clean;

they are mechanical but load-bearing, and their failure modes are instructive. The classes that cost the most time:

A64/C64 interworking. Morello executes in two instruction-set modes; purecap code runs in C64. An indirect call through an *integer* register (`blr x`*n* rather than `blr c`*n*) does not switch mode, and—the trap within the trap—setting the link address's low bit does *not* set `PSTATE.C64`: the callee then decodes C64 prologue bytes as A64 garbage. Every interpreter codelet that calls into the VM through a raw function address had to be routed through capability branches.

Capability-blind code generation. Patterns that are correct on aarch64 silently strip tags on Morello: a conditional select (`csel`) on what is actually a capability value, integer compare-and-move on 16-byte object headers, an integer store overwriting half a capability. The 16-byte mark word also forces the header lock CAS to be a capability-sized CAS.

DDC-relative assembly. In purecap CheriBSD the default data capability is null; hand-written `.S` code that addresses memory as `[x0]` (DDC-relative) faults even though the "pointer" is valid. HotSpot's `SafeFetch` stubs were the prominent case.

Signal-handler composition. The GC trap handler, HotSpot's expected-fault machinery, and any diagnostic crash handler all receive the same signal; each must try *all* the others' redirects (SafeFetch continuation, heal path) before declaring a fault fatal. Getting this wrong turns a healable stale-reference trap into a VM abort that looks like GC corruption.

VM-internal raw references. HotSpot's C++ code caches object addresses in places the root scan does not cover (interned handles, scratch fields). These never fault when merely *copied* as values—only dereferences trap—so a stale cached reference can be compared, hashed, or stored long before the barrier can intervene. We bisected aggressive-relocation boot failures to this class (§6); it is the purecap analogue of the classic "unreported root" GC bug, made subtler because reads-as-values escape the hardware check.

The collection cycle itself runs as a safepoint `VM_Operation`; an optional concurrent thread triggers cycles on a timer and is stable under light load (§6). Relocation volume per cycle is capped by a tunable move limit; integrity tests also run with the limit effectively removed (every *directly root-referenced* object relocated, 1,016 in our largest configuration—transitively reachable objects stay in place, as there is no marking traversal) and verify structure and reference identity afterwards.

5 Evaluation

5.1 Setup and method

All measurements run on purecap CheriBSD 15.0-CURRENT (`GENERIC-MORELLO-PURECAP`, `aarch64c`) under QEMU system-mode emulation of Morello, with the OpenJDK 17 template interpreter (`-Xint`), compressed oops and compressed class pointers disabled, and the concurrent collector triggering cycles every 50 ms with a per-cycle move limit of 8 objects unless stated. Phase timings are taken inside the safepoint operation with the VM's nanosecond clock; they cover the operation's execution and exclude safepoint synchronisation and resume, so true end-to-end pauses are slightly larger; barrier-side counters (heal faults, heal time, identity slow-path executions) are accumulated in the signal handler with a monotonic clock and reported at VM exit.

What emulation does and does not invalidate. QEMU inflates all absolute times and does not model Morello's microarchitecture, so we make no absolute-latency claims. The claims we do make are structural: *which* costs scale with *what* (heap size, root count, batch size), and those relationships are properties of the algorithm and the revocation mechanism, not of the emulator. Workloads are purpose-built: an allocation-heavy benchmark whose live set grows continuously (StoplessBench), a linked-structure integrity test that relocates every directly root-referenced

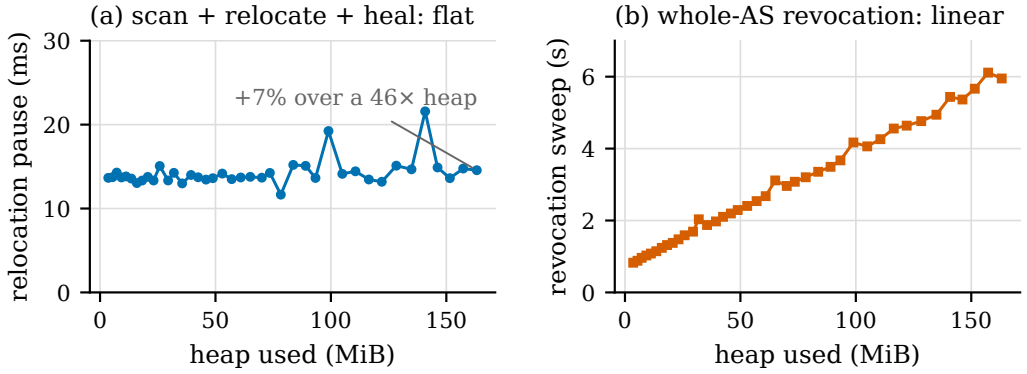

Fig. 2. StoplessBench, root set constant at 894 after warm-up, heap footprint growing 3.6 → 163 MiB (46×); 40 collection cycles, 8 objects relocated per cycle. (a) The mutator-visible relocation phase (root scan + copy + root fixup) stays at 13–15 ms. (b) The `cheri_revoke` sweep grows linearly with mapped memory, 0.83 → 5.95 s. Absolute times are QEMU-inflated; the claim is the contrast in shape.

object and then verifies structure and reference identity (IntegrityGC), and a string-concatenation workload that exercises `invokedynamic`, `MethodHandle` bootstraps, and identity-keyed caches (ConcatTest). They are microbenchmarks, not DaCapo; running standard suites requires JIT and library coverage beyond any current purecap JVM, including ours.

5.2 The relocation pause is independent of heap size

Figure 2 is the central result. StoplessBench grows its allocated footprint continuously while its root-set size stabilises, isolating heap size as the variable. Over a 46× heap range at a constant 894 roots, the relocation pause moves from 13.7 ms to 14.6 ms comparing endpoints (+7%; individual cycles range 11.7–21.6 ms with no trend in heap size), because the pause does work proportional to root slots scanned plus *bytes copied*—never to heap size. (The experiment holds both fixed: 8 small objects per cycle. A root holding a giant array would grow the copy term; the bound is roots + relocated bytes, not roots alone.) Part of this flatness is by construction—the workload holds roots and per-cycle relocation volume fixed—and that is the point of the experiment: it is a negative control showing the pause contains *no hidden heap-proportional term*. There is no marking phase, no per-object liveness traversal, and no card or remembered-set scan; a collector design that secretly depended on heap size would show it here. The sweep, which is heap-proportional, also currently executes inside the safepoint and is therefore pause time too—panel (b) and §5.4 quantify it; making it concurrent (as Cornucopia Reloaded does for C heaps) is the obvious next step and is orthogonal to the relocation path measured in panel (a). By contrast the sweep itself, panel (b), is linear in mapped memory—0.83 s at 3.6 MiB to 5.95 s at 163 MiB—confirming that sweeping, exactly as in the C/C++ revocation literature [7, 17], is the cost to engineer around, and motivating §5.4.

5.3 Cost of the barrier mechanisms

Table 1 summarises. The in-handler heal work—register-file scan, transitive forwarding lookup, capability rebuild, register patch—averages 22.2 µs over 871 heals in IntegrityGC and 17.9 µs over 417 heals in ConcatTest (these runs include the transitive chase; pre-fix one-hop lookups measured 7–9 µs). (The timer brackets the handler body only: kernel signal delivery and `sigreturn` are outside it, so the full trap round trip is strictly larger—a measurement to redo with hardware performance

Table 1. Measured costs of the three mechanisms (QEMU-emulated; relative magnitudes are the claim). Heal statistics from IntegrityGC and ConcatTest under the concurrent collector.

mechanism	cost	scales with	frequency
relocation pause	13–15 ms	root slots + relocated bytes	per cycle
revocation sweep	0.83–5.95 s	mapped memory	per cycle (or batch)
heal (handler body)	18–22 µs	—	per *used* stale ref
identity slow path	2 table lookups	—	153–571 / run

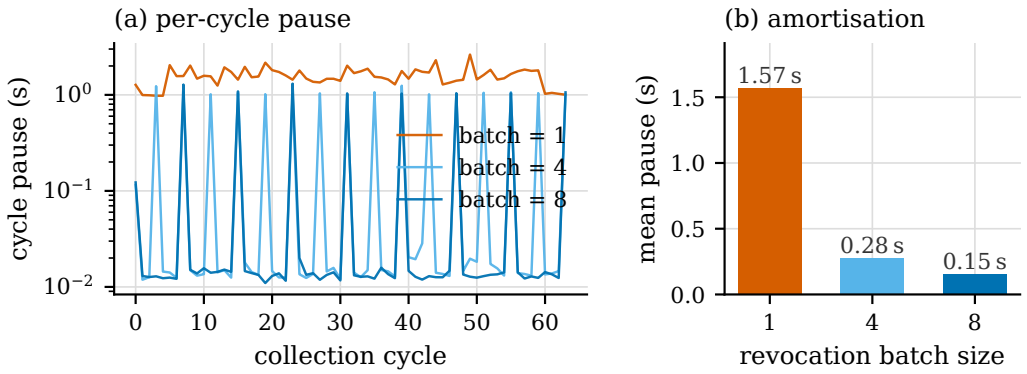

Fig. 3. Batched revocation, IntegrityGC, workload-constrained configuration (64 benchmark-driven STW cycles after bootstrap per batch size, fixed implementation; all 3 × 64 integrity verifications passed; full logs in the data directory). (a) Per-cycle pause, log scale: with batching most cycles pause only for relocation (~11 ms floor); the sweep cost concentrates in every Nth cycle. (b) Mean pause per cycle falls from 1.57 s to 0.15 s (observed 10.3×; part beyond the ideal 8× is cross-run sweep-amplitude variance). Batching gives no worst-case improvement—moving the sweep off-pause is the CLG design assessed feasible in §5.4.

counters on a board.) Aggregate heal time is 7–19 ms per run—one-and-a-half to two orders of magnitude below a single sweep—supporting the design's bet that lazy, per-used-reference healing is cheap. The mutator's load fast path, to restate, contains no added instructions at all.

5.4 Batched revocation amortises the dominant cost

Because the sweep visits all memory regardless of marks, sweeping every Nth cycle divides its amortised cost by N while relocation continues every cycle. We measure this in a *workload-constrained configuration*: each collection is a stop-the-world cycle driven explicitly by the benchmark after VM bootstrap (the mutator runs between cycles but performs no reference comparisons on, and no mutation of, relocated objects inside the batch window), on the fixed (transitive-chase) implementation; one 64-cycle trace per batch size. Figure 3: mean pause per cycle falls 1.57 → 0.28 → 0.15 s for batch sizes 1, 4, 8—an observed 10.3× reduction, tracking the sweep count (64/16/8); all 3 × 64 integrity verifications passed. (Per-sweep spike amplitude varies across the three runs—means 1.53/1.04/1.10 s, maxima 2.63/1.25/1.31 s—so part of the ratio beyond the ideal 8× is cross-run variance; we have not repeated the runs for error bars.) The per-cycle trace shows the expected shape: a floor at the heap-independent relocation pause (~11 ms) with the sweep cost concentrated in every Nth cycle. Batching provides no algorithmic worst-case improvement: the sweep cycle still pays the full heap-linear scan.

The same mechanism run *concurrently* demonstrates the window unsoundness of §3.5 empirically: with the background collector, batch =32 completed 1 of 6 trials and batch =8 0 of 4 (batch =1: stable), failing with symptoms consistent with the identity mechanism—`IllegalAccessError` out of `java.lang.invoke` during bootstrap, whose identity-keyed caches compare references inside the window. Batched revocation is sound only when the window is closed to identity-sensitive and mutating code.

Feasibility of the sound design. The instrument that closes the window exists in our stack, and we verified it works under QEMU emulation: CheriBSD's load-side revocation drives Morello's per-page capability-load generation (CLG) barrier, and a minimal probe shows that (i) *opening* a revocation epoch—after which no stale capability can be loaded with its tag intact—costs 3.7 ms with no data scan; (ii) the first capability load from a not-yet-scanned page takes a CLG fault and the kernel lazily sweeps that page (~0.17 ms per first-touch page, probed across a 16 MiB capability-dense region); (iii) the kernel API additionally offers a worker-thread vmspace scan (`CHERI_REVOKE_ASYNC`; the probe confirmed only its flag constraints—it is rejected in combination with the closing flag—so asynchronous operation is an API capability we have not yet exercised). The sound batched design follows: relocate and mark each cycle; *open* the epoch inside the pause (milliseconds, heap-independent); run the heap-linear scan asynchronously off-pause. The mutator pause becomes relocation plus epoch-open—both heap-independent—which is the Cornucopia Reloaded result transplanted to a moving collector. Building this is the immediate next step.

5.5 The identity barrier's slow path is rare

Over a full ConcatTest run—chosen because `java.lang.invoke` bootstrap is dense with identity-keyed caching—the identity barrier's slow path executed 153 times, against 417 heal faults (IntegrityGC: 571 slow-path executions, 871 heals; raw logs in the data directory). The fast-path cost is two tag-check instructions and a branch by construction; we report slow-path *counts* only and have not measured fast-path throughput impact (interpreter dispatch dominates under `-Xint`; a JIT-relative measurement is future work).

6 Discussion and limitations

Emulation. We re-emphasise: all absolutes are QEMU-scaled. On hardware, signal delivery, memory bandwidth (the sweep is bandwidth-bound [17]), and the relocation copy all shrink by different factors; the flat-versus-linear contrast and the batching arithmetic are preserved because they are counting arguments, not timings. Reproducing Figure 2 on a Morello board is the natural next step.

Stop-the-world relocation. The measured configuration relocates at safepoints; concurrency in StoplessGC currently means the *collector schedule* (a background thread triggering cycles, the mutator running between them, healing on demand). True concurrent relocation—mutators running *during* the copy—needs an object-granularity copy protocol; the hardware trap handles the post-move window already, and revocation's atomic global cut-over is precisely the property that makes the post-copy side tractable.

Coverage of VM-internal references. Our root scan covers the standard strong-root sets. HotSpot additionally caches raw object addresses in VM-internal C++ state that is invisible to the root scan and—because capability *reads* do not trap, only dereferences—can launder a stale reference back into circulation. We hit this as boot-time failures under aggressive relocation and bisected it to specific unscanned caches. The fix is the classic one (enumerate or eliminate the caches); we flag it because the hardware barrier's guarantee is easily over-read: it covers *dereference*, not *data flow*.

Interpreter only. Like all current purecap JVM work to our knowledge [10, 14], we run interpreted. Template-interpreter bytecodes are exactly where our barrier costs sit (the `acmp` tiers), so JIT changes the constant factors, not the structure.

Not yet a complete collector. Liveness, reclamation, and address reuse remain unimplemented: relocation candidates come from the root scan rather than a marking traversal, dead objects are never collected, and the bump arena never recycles addresses (which also sidesteps the forwarding-generation ambiguity of §3.2). Composing a marker and an evacuating reclaimer with this substrate uses standard machinery; the open CHERI-specific question is the reuse invariant—when has every stale capability into a region provably been healed or destroyed—for which the revocation epoch counter itself looks like the right primitive.

Memory overhead. The prototype never reclaims: the bump arena retains old copies indefinitely (revocation invalidates capabilities; it does not free memory), and forwarding entries (three machine words per relocation) accumulate. In a completed design with reclamation, batching would add float bounded by relocated bytes per window; today the float is unbounded by construction, which is part of the "not yet a collector" boundary.

7 Related work

CHERI revocation. CHERIvoke [17], Cornucopia [6], and Cornucopia Reloaded [7] build and optimise sweeping revocation for C/C++ temporal safety; revoked references are use-after-free errors, terminal by design. We inherit their mechanism (quarantine, shadow bitmap, sweep) and invert the trap's meaning, from verdict to redirect. Cornucopia Reloaded's per-page load barrier is also the missing piece for closing our batching window, a reunion of the two lines we find pleasing.

Hardware-assisted read barriers. Appel–Ellis–Li [1] used page protection as the barrier; its page granularity caused multi-object stalls and its protection flips are TLB-expensive. Capability revocation gives *per-object* (indeed per-capability) granularity with one global cut-over. Azul Vega [5] put the barrier in a custom instruction, achieving per-reference granularity at the price of bespoke silicon; CHERI offers the trap-on-stale primitive in a general-purpose, security-motivated architecture.

Software barriers in production JVMs. ZGC [11, 18] and Shenandoah [8] demonstrate that software self-healing barriers can reach sub-millisecond pauses on commodity hardware, at a per-load instruction cost and, for ZGC, a pointer encoding that purecap disallows outright (§2.4). C4 [13] likewise relies on address-space tricks (virtual-memory remapping) that capability bounds constrain. Our contribution is not to outperform these collectors but to show the barrier they implement in software is already present, exact, and free-per-load in capability hardware.

Java on CHERI.. MOJO [14] ports OpenJDK 17's Epsilon, Serial, and G1 to CheriBSD/Morello; Lowther et al. [10] optimise a MicroPython interpreter on Morello. Neither moves objects under a hardware barrier. Bramley et al. [3] study CHERI `malloc` implementations, the non-moving end of the same design space.

8 Conclusion

Capability revocation was built to make dangling pointers trap. A moving garbage collector needs exactly that trap—plus a forwarding table on the other side of it. StoplessGC demonstrates the combination end to end in OpenJDK on Morello: relocation pauses bound by roots rather than heap (+7% pause across a 46× heap), lazy per-use healing measured in microseconds, an identity barrier that almost never takes its slow path, and a sweep cost that batches away 10.3× in a controlled

configuration because the sweep's cost is approximately independent of how much it revokes—with the per-page load-barrier instrument that makes batching sound verified working under our stack. The capability-specific lessons—integer-only GC metadata rebuilt from revocation-exempt roots, identity as the one barrier hardware cannot give you, reads-as-values as the residual coverage gap—are, we believe, durable properties of building moving collectors on this class of hardware, and the remaining engineering (per-page load barriers for the batch window, concurrent copy, JIT) has known designs waiting on the same substrate.

AI contribution statement. The design, implementation, debugging, measurement, and manuscript of this work were produced by Claude (Anthropic)—Opus 4.8 and Fable 5 models, operating via Claude Code—under the direction, constraint setting, and review of the human author. The division of labour is recorded in the project repository's development log. Per current arXiv and publisher policies, AI systems are not listed as authors because authorship carries accountability; the human author assumes that accountability and verified the empirical results and every citation.

References

[1] Andrew W. Appel, John R. Ellis, and Kai Li. 1988. Real-Time Concurrent Collection on Stock Multiprocessors. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. doi:10.1145/53990.54005

[2] Henry G. Baker. 1978. List Processing in Real Time on a Serial Computer. *Commun. ACM* 21, 4 (1978), 280–294. doi:10.1145/359460.359470

[3] Jacob Bramley, Dejice Jacob, Andrei Lascu, Jeremy Singer, and Laurence Tratt. 2023. Picking a CHERI Allocator: Security and Performance Considerations. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on Memory Management (ISMM)*. doi:10.1145/3591195.3595278

[4] Rodney A. Brooks. 1984. Trading Data Space for Reduced Time and Code Space in Real-Time Garbage Collection on Stock Hardware. In *Proceedings of the ACM Symposium on LISP and Functional Programming (LFP)*. doi:10.1145/800055.802042

[5] Cliff Click, Gil Tene, and Michael Wolf. 2005. The Pauseless GC Algorithm. In *Proceedings of the 1st ACM/USENIX International Conference on Virtual Execution Environments (VEE)*. doi:10.1145/1064979.1064988

[6] Nathaniel Wesley Filardo, Brett F. Gutstein, Jonathan Woodruff, Sam Ainsworth, Lucian Paul-Trifu, Brooks Davis, Hongyan Xia, Edward Tomasz Napierala, Alexander Richardson, John Baldwin, David Chisnall, Jessica Clarke, Khilan Gudka, Alexandre Joannou, A. Theodore Markettos, Alfredo Mazzinghi, Robert M. Norton, Michael Roe, Peter Sewell, Stacey Son, Timothy M. Jones, Simon W. Moore, Peter G. Neumann, and Robert N. M. Watson. 2020. Cornucopia: Temporal Safety for CHERI Heaps. In *IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP40000.2020.00098

[7] Nathaniel Wesley Filardo, Brett F. Gutstein, Jonathan Woodruff, Jessica Clarke, Peter Rugg, Brooks Davis, Mark Johnston, Robert Norton, David Chisnall, Simon W. Moore, Peter G. Neumann, and Robert N. M. Watson. 2024. Cornucopia Reloaded: Load Barriers for CHERI Heap Temporal Safety. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2*. doi:10.1145/3620665.3640416

[8] Christine H. Flood, Roman Kennke, Andrew Dinn, Andrew Haley, and Roland Westrelin. 2016. Shenandoah: An Open-Source Concurrent Compacting Garbage Collector for OpenJDK. In *Proceedings of the 13th International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools (PPPJ)*. doi:10.1145/2972206.2972210

[9] Richard Grisenthwaite, Graeme Barnes, Robert N. M. Watson, Simon W. Moore, Peter Sewell, and Jonathan Woodruff. 2023. The Arm Morello Evaluation Platform—Validating CHERI-Based Security in a High-Performance System. *IEEE Micro* 43, 3 (2023), 50–57. doi:10.1109/MM.2023.3264676

[10] Duncan Lowther, Dejice Jacob, and Jeremy Singer. 2023. CHERI Performance Enhancement for a Bytecode Interpreter. *arXiv preprint arXiv:2308.05076* (2023).

[11] OpenJDK Project. 2023. JEP 439: Generational ZGC. https://openjdk.org/jeps/439.

[12] Filip Pizlo, Daniel Frampton, Erez Petrank, and Bjarne Steensgaard. 2007. Stopless: A Real-Time Garbage Collector for Multiprocessors. In *Proceedings of the 6th International Symposium on Memory Management (ISMM)*. doi:10.1145/1296907.1296927

[13] Gil Tene, Balaji Iyengar, and Michael Wolf. 2011. C4: The Continuously Concurrent Compacting Collector. In *Proceedings of the International Symposium on Memory Management (ISMM)*. doi:10.1145/1993478.1993491

[14] THG and University of Manchester. 2024. MOJO: Porting OpenJDK 17 and its Garbage Collectors to CheriBSD on Morello. Project site: https://www.mojo-jvm.org. Ported collectors: Epsilon, Serial, G1.

[15] University of Cambridge and SRI International. 2026. CheriBSD: A Capability-Adapted Version of FreeBSD. https://www.cheribsd.org.

[16] Robert N. M. Watson, Peter G. Neumann, Jonathan Woodruff, Michael Roe, Hesham Almatary, Jonathan Anderson, John Baldwin, Graeme Barnes, David Chisnall, Jessica Clarke, et al. 2023. *Capability Hardware Enhanced RISC Instructions: CHERI Instruction-Set Architecture (Version 9).* Technical Report UCAM-CL-TR-987. University of Cambridge, Computer Laboratory.

[17] Hongyan Xia, Jonathan Woodruff, Sam Ainsworth, Nathaniel W. Filardo, Michael Roe, Alexander Richardson, Peter Rugg, Peter G. Neumann, Simon W. Moore, Robert N. M. Watson, and Timothy M. Jones. 2019. CHERIvoke: Characterising Pointer Revocation Using CHERI Capabilities for Temporal Memory Safety. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO).* doi:10.1145/3352460.3358288

[18] Albert Mingkun Yang and Tobias Wrigstad. 2022. Deep Dive into ZGC: A Modern Garbage Collector in OpenJDK. *ACM Transactions on Programming Languages and Systems* 44, 4 (2022), 22:1–22:34. doi:10.1145/3538532